K-GetNID: Knowledge-Guided Graphs for Early and Transferable Network Intrusion Detection

Minxiao Wang, Ning Yang, *Member, IEEE*, and Ning Weng, *Senior Member, IEEE*

***Abstract*— Developing early and transferable Network Intrusion Detection Systems (NIDSs) is essential for robust network security. Early detection prevents further damage, while transferable NIDSs enables reuse across diverse networks. For Machine Learning (ML) and Deep Learning (DL)-based NIDSs, transferability significantly reduces data collection and annotation costs for timely attack mitigation. Current DL-based early intrusion detection studies often focus on identifying attacks from the first few packets, neglecting the crucial aspect of adjustable early detection. Additionally, most DL-based NIDS methods overlook transferability during both design and evaluation phases. To address these limitations, we propose K-GetNID, a knowledge-guided graph learning-based NIDS that excels in both early and transferability. We introduce a Heterogeneous Temporal Graph (HTGraph) to represent the dynamic feature series of network flows, providing enough information for early detection. Additionally, we construct this HTGraph format based on prior knowledge about feature types and correlations to assist the neural network in learning general and transferable knowledge for intrusion detection. We develop a corresponding Heterogeneous Temporal Graph Neural Network (HTGNN) model to learn from the HTGraph format. Furthermore, an Adjustable Early Detection Decoder is designed to enhance the generalization of the proposed model to the input distribution shifts caused by early detection. Experiments on CIC-IDS-2017 and UNSW-NB15 datasets show that K-GetNID matches the performance of deep learning methods, excelling in adjustable early intrusion detection and transferability.**

***Index Terms*— Intrusion detection systems, graph neural networks, transferability, early detection, deep learning.**

I. Introduction

EARLY and transferable NIDSs play a crucial role in addressing the challenges posed by the dynamic and diverse nature of network environments for ensuring network security. An investigation into potential solutions for network intrusion detection suggests that NIDSs utilizing ML and DL models could prove highly effective [1], [27]. However, existing ML/DL models still encounter limitations, pose open questions, and face unique challenges in the context of NIDS tasks [30], [33], such as the diversity of traffic caused by different network environments and the fixed and biased parameter selection during the training stage may not be suitable for the deployment scenario.

For transferability, given the broad application scenarios of NIDSs, a DL-based NIDS model trained for one scenario should be easily adaptable or fine-tuned for another. However, the high cost associated with collecting and annotating a NIDS dataset for each scenario is often unaffordable. In the context of early intrusion detection, a proficient NIDS model should demonstrate the ability to detect intrusions early and generalize to distribution shifts in inputs. Early intrusion detection involves discerning whether network traffic flows are benign or malicious in the initial stages, leading to changes in input patterns and providing only a partial view of the original input traffic. These alterations in input significantly degrade the inference accuracy of trained NIDS models.

Although early intrusion detection challenges current DL-based NIDSs, it is essential so that correct counterattack responses can be implemented before an attack can inflict more damage [3], [29]. Early intrusion detection also offers additional benefits, such as saving time in collecting raw packets and extracting features. Additionally, we observe that many existing DL-based NIDS models [4], [13], [21], [23], [29] could be capable of early intrusion detection if trained with early detection data, and their early detection accuracy is only slightly degraded [36]. Therefore, the real challenge in early intrusion detection lies in the trade-off between accuracy and response speed, which cannot be further adjusted after the training stage. In particular, the trained NIDS models are not generalized to distribution shifts caused by early detection [35].

Learning early and transferable NIDS models requires these models to identify attack behavior patterns that remain consistent across varying application scenarios and input domains. Consequently, we contend that training models to capture the correlations among numerous input items (such as features in tabular format) is more crucial than learning the absolute values of independent input items. While hand-picked features, grounded in extensive domain knowledge from network security experts, significantly contribute to distinguishing different types of traffic. However, the use of a tabular format in feature-based methods poses a challenge due to the existence of heterogeneous and sparse relationships among feature values'a current open research question [19]. Utilizing Multi-Layer Perceptron (MLP)-based models, which are fully connected, is difficult to learn feature correlations

Manuscript received 22 December 2023; revised 24 June 2024; accepted 16 July 2024. Date of publication 22 July 2024; date of current version 29 July 2024. This work was supported in part by Dr. Yang's SIUC Startup and NSF under Award 2018919. The associate editor coordinating the review of this article and approving it for publication was Dr. Z. Berkay Celik. *(Corresponding author: Ning Yang.)*

Minxiao Wang and Ning Weng are with the Computer Engineering Program, School of ECBE, Southern Illinois University Carbondale, Carbondale, IL 62901 USA (e-mail: minxiao.wang@siu.edu; nweng@siu.edu).

Ning Yang is with the Information Technology Program, School of Computing, Southern Illinois University Carbondale, Carbondale, IL 62901 USA (e-mail: nyang@siu.edu).

Digital Object Identifier 10.1109/TIFS.2024.3431932

from scratch [2]. Furthermore, the tabular format fails to provide temporal information, rendering tabular-based NIDS methods insufficiently generalized to handle dynamic inputs, such as early intrusion detection.

The recent state-of-the-art tabular model FT-transformer [12] improves the performance of deep models on tabular data by separately embedding numerical and categorical features and modeling the embeddings with the Transformer model. Additionally, the FT-transformer model demonstrates effectiveness in the NIDS field [37]. Splitting different types of features and modeling the feature embeddings with the self-attention mechanism [34] addressed the mentioned heterogeneous and sparse relationship issues but not tackle the lack of temporal information. Feature series-based methods can provide temporal information to NIDS, but extending the temporal dimension increases the size of each data sample significantly compared to the tabular format. Consequently, applying the Transformer in feature series format incurs high time complexity and memory usage. Despite efforts to enhance the efficiency of Transformers on time series, as seen in models like Informer [44], performing self-attention on a large scale of multivariate time series (multiple feature series) remains inefficient. Recognizing the limitations of tabular and feature series formats has motivated the design of a new type of NIDS data format that incorporates prior knowledge to overcome these shortcomings.

An alternative data format to tabular and feature series is the graph structure, as both provide numerical and relational information, allowing the incorporation of prior knowledge. For example, provenance-based studies against Advanced and Persistent Threats (APTs) such as UNICORN [14] and HOLMES [24], use graph structures to analyze the correlation among actions or events for detecting attacks. For example, in the NIDSs on enterprise networks, graph embedding learning [18], [28] show the effectiveness for detecting lateral movement of APTs. Graph formats and graph neural networks possess the capability to explicitly capture inter-temporal and inter-variable relationships, a challenge that traditional and other deep neural network-based methods frequently struggle with [17]. However, to the best of our knowledge, there is no existing graph-based NIDS that is flow-based. Drawing inspiration from the successful application of HTGraph in other fields [10], [22], this paper investigates the use of HTGraph to represent network flow for NIDS design.

In this paper, we propose a novel framework, as shown in Fig. 2, which incorporates prior knowledge into a NIDS design using heterogeneous temporal graphs and neural network models. The proposed heterogeneous graph data format is utilized to represent each network flow, incorporating high-level prior knowledge such as feature types and dependence between features. Additionally, we extend the heterogeneous graph to a HTGraph by recording dynamically updated features. For this HTGraph format, we design a corresponding HTGNN-based NIDS model.

Furthermore, edges based on prior knowledge can reduce the computational load in feature series format by narrowing the self-attention calculation space from global to the predefined edge range. The results demonstrate that the proposed HTGNN model, trained with attention restricted by the graph format, achieves better transferability than existing methods. To enable adjustable early intrusion detection in the inference stage, we further designed an Adjustable Early Detection Decoder to train the HTGNN model for generalization to input distribution shifts caused by early detection. The main contributions are summarized as follows:

1) **Novel NIDS traffic representation format and detection model**: developed a comprehensive NIDS system by integrating a HTGraph with an HTGNN-based NIDS model. The HTGraph format includes edges that capture feature correlations and temporal dependencies, while the HTGNN aggregates feature information through the pre-defined edges based on prior knowledge.
2) **Improving NIDS transferability**: the proposed HTGraph format and HTGNN NIDS model leverage prior knowledge to guide modeling the correlations among input elements, enhancing attention calculation efficiency. The results of transfer learning demonstrate that the simplified attention-based model exhibits better transferability.
3) **Enhancing generalization to early intrusion detection**: the designed Adjustable Early Detection Decoder enables the proposed HTGNN model to generalize to the distribution shifts caused by early detection. The results demonstrate that the proposed NIDS model can still maintain its detection accuracy under distribution shifts in the inference stage.

In the remainder of this paper, we introduce related works in Sec.II and preliminary in Sec.III. We describe the entire pipeline of our proposed framework in Sec.IV, including network flow HTGraph construction, the HTGNN-based NIDS model, and the Adjustable Early Detection Decoder. In Sec.V, we present the evaluation results and analysis. Finally, we provide our conclusion in Sec.VI.

II. Related Works

This section introduces feature-based NIDSs and the graph-based NIDSs. We also summarize the similarities and differences between our work and existing works.

A. Feature-Based Network Intrusion Detection

Nowadays, flow-based NIDSs typically consists of a preprocessing module that converts traffic flows into samples with a particular data format and a detection module that predicts classes of traffic flows based on the format [25], [39], [43]. This field is predominantly dominated by the tabular data format, specifically the feature vector. Feature-based methods [38] extract pre-defined features from traffic flows and feed these features into neural network models. However, the tabular format lacks temporal information, making it unsuitable for adjustable early intrusion detection. Besides the tabular format, another existing feature-based format is the feature series. He et al. [15] introduced a Multimodal-Sequential data format in their work. Doriguzzi-Corin et al. developed a sampling window-based packet sequences data

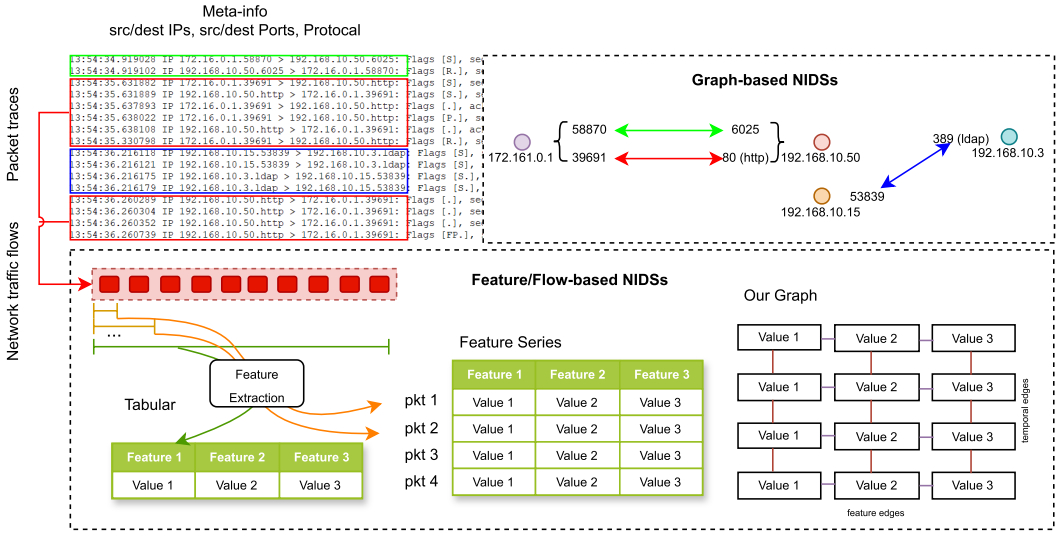

Fig. 1. The different NIDS data formats. The network traffic can be constructed in different formats to leverage the different DL models and different types of information in them. This figure presents the differences among Tabular, Feature Series, existing topology graph, and the proposed graph.

TABLE I

COMPARISON WITH EXISTING FEATURE-BASED NETWORK INTRUSION DETECTION METHODS

Paper	Format	Model	Correlations	Benefits[1]
[25]	Tabular	DAE/MLP	Feature	N/A
[15]	Tabular	MDAE/LSTM	Feature	N/A
[38]	Tabular	CNN/LSTM	Feature	N/A
[7]	Feature Series	CNN	Temporal	Early
Our	Feature Graph	HTGNN	Temporal & Feature	Early & Transferable

[1] In the table header, the columns "Benefits" only refer to whether the method can improve early detection or transferability.

TABLE II

COMPARING ADOPTED GRAPH STRUCTURE WITH EXISTING GRAPH-BASED NETWORK INTRUSION DETECTION METHODS

Paper	Graph Type	Node	Edge	Message Passing
[32]	Client-server	Packet	Adj relation	Packet size & direction
[41]	Communication	Host+port	Traffic	Statistical features
[45]	Topology	Host+port	Traffic	Statistical features
[20]	Topology	Host+port	Traffic	Statistical features
[8]	Dynamic Topology	Host	Traffic	Statistical features
[16]	Client-server	Packet	Adj relation	Packet size & direction
[40]	Topology	Host	Traffic	NetFlow feature
Our	Dynamic features	Single Feature	Feature relation	Single traffic behavior

format for DDoS detection in their study [7]. While the feature series format is similar to the feature vector as they both use a pre-defined feature set, the feature series format also records the dynamic trends of all the features as packets arrive. Feature series-based methods typically adopt learning methods, such as Transformer and Long Short-Term Memory (LSTM), which are also used in the field of time series learning.

B. Graph-Based Network Intrusion Detection

In recent years, there has been a significant amount of research focused on NIDSs that utilize graph analysis. Shen et al. [32] introduce the Traffic Interaction Graph (TIG) as a comprehensive representation of encrypted DApp flows, incorporating multidimensional features within bidirectional client-server interactions. Yao et al. [41] design spatial-temporal deep learning on network communication graphs, using the graph structure as the additional prior information. Zola et al. [45] propose a pipeline for working with network traffic data, extracting temporal graph-based (node) features and finally detecting malicious nodes (IPs). Lo et al. [20] propose E-GraphSAGE, a GNN approach that allows capturing both edge features of a graph as well as the topological information for network intrusion detection in IoT networks. Duan et al. [8] propose a Dynamic Line Graph Neural Network (DLGNN)-based intrusion detection method with semi-supervised learning. The DLGNN extracts dynamic lines (edges) features by learning from flow statistics features in each snapshot. Hu et al. [16] propose a graph embedding method to model the packet interactions in network traffic for NIDS.

Most existing graph-based NIDS convert network traffic flows into a dynamic IP address graph. Their primary advantage lies in the ability of the graph structure to incorporate the "global" topology of cyberspace and provide insights into the evolutionary patterns of host-to-host communication interactions. These graph-based methods focus on the global view of the interaction among hosts and classify whether graph nodes (hosts) behave maliciously.

C. Differences From Existing Work

As shown in Fig. 1, most of the existing Graph-based NIDS works represent the hosts in the network as the nodes of the graph, and the communication between two hosts is represented as an edge. The packet trace records or the network traffic flow records are considered as the interactions among all nodes in the graph along the time dimension. In our method, we define the dynamically updated features in the Feature Series format as the nodes of the graph. When only one packet arrives, our graph has N nodes to represent the N different features. For each arriving packet, the graph will add N nodes to represent the updated features. In our graph, the edges connect nodes along the feature's spatial and temporal dimensions.

TABLE III
SUMMARY OF SYMBOLS AND DESCRIPTIONS

Symbols	Definitions and Descriptions
x	feature types based on the feature's value scale and unit
i	number of features that belong to one specific type
t	the timestamp
$f^x_{i,t}$	flow's x type ith dynamic statistics feature at time t
$HTGraph_{flow}$	heterogeneous temporal graph for representing a flow
$V^x = \{v_{i,t}\}$	the set of nodes belonging to type x
$E^{homo} = \{e^x_{(i,j),t}\}$	the set of edges connect pairs of nodes from the same type at the same time
$E^{heter} = \{e^{x_1,x_2}_{(i,j),t}\}$	the set of edges connect pairs of nodes from different types at the same time
$E^{temp} = \{e^x_{i,(t,t+1)}\}$	the set of edges connect pairs of temporally adjacent nodes from the same feature
$h^x_{i,t}$	learned embedding for feature nodes $v_{i,t}$
θ	model's parameters

Although our work also adopts a graph structure, it should be noted that our method is fundamentally different from the existing graph-based NIDS methods [16], [20]. Instead of utilizing network topology information or packet-sending interactions, K-GetNID employs a heterogeneous temporal graph to model different correlations among dynamic feature series. Therefore, K-GetNID is a feature-based NIDS method to represent flow behaviors and classify the entire graph as benign or malicious flow. Graph formats and graph neural networks have the capability to explicitly model inter-temporal and inter-variable relationships, a task that traditional and other deep neural network-based methods often find challenging [17].

III. PRELIMINARY

In this section, we introduce the concepts related to using HTGraphs to formalize the NN application for flow-based NIDS. Specifically, an HTGraph is generated to represent a network traffic flow for the GNN-based NIDS model to classify whether the flow is malicious or not. The expert-defined heterogeneous features, dynamically extracted with the flow's packets arriving, are included as nodes in the graph. The correlation among different features and the sequential relationships of the same feature are represented by spatial and temporal edges of an HTGraph. All defined notations are summarized in Table III.

Following the definition in [42], the heterogeneous graph can be defined as $G = (V, E, O_V, R_E)$, where V represents nodes with different types, E represents edges, O_V is the set of object types and R_E is the relation types.

Definition: Given a network flow $flow$, which consists of a series of packets $pkt_1, pkt_2, \ldots, pkt_n$, statistical features $(f_1, f_2, \ldots, f_m)_t$ are dynamically updated for NIDS prediction. Our goal is to predict $flow$ is malicious or benign at duration T through learning on a heterogeneous temporal features graph $HTGraph_{flow}(T)$, which is constructed from dynamic statistical features as given in Equation (1),

$$HTGraph_{flow}(T) = (V^x, E^{homo}, E^{heter}, E^{temp}) \quad (1)$$

where x refers to feature types, V^x refers to the set of nodes belonging to type x, E^{homo} refers to the edges connect pairs of homogeneous nodes, E^{heter} refers to the edges connect pairs of heterogeneous nodes, E^{temp} refers to the temporal edges.

Our framework aims to embed the heterogeneous and weak correlated features f into representation vectors $e \in \mathbb{R}^d$, where d is the dimension of embedding vectors. Therefore, the embedding model $\mathcal{G}^{embed}$ seeks to learn parameters θ_1 as follows:

$$\mathcal{G}^{embed}_{\theta_1}(HTGraph_{flow}(T, f^x_{i,t})) \rightarrow h^x_{i,t} \quad (2)$$

where $f^x_{i,t}$ refers to a flow's x type ith dynamic statistics feature at time t.

Another model $\mathcal{G}^{pred}$ aims to learn parameters θ_2 to predict whether the flow is malicious or not, as given in Equation (3). The prediction is based on the updated HTGraph, in which nodes' representations are replaced by embedding vectors $h^x_{i,t}$ obtained from Equation (2),

$$\mathcal{G}^{pred}_{\theta_2}(HTGraph_{flow}(T, h^x_{i,t})) \rightarrow prediction \quad (3)$$

where $h^x_{i,t}$ refers to the learned embedding for feature nodes $v_{i,t}$.

IV. PROPOSED FRAMEWORK

In this section, we introduce the design of our proposed framework. First, we present the overview structure of the whole framework in Sec. IV-A. We describe how to construct a heterogeneous temporal graph format to represent network traffic in Sec. IV-B. Then we illustrate the design of heterogeneous temporal graph neural networks for NIDS in Sec. IV-C. Finally, we delve into the details of the Adjustable Early Detection Decoder in Sec. IV-D.

A. Framework Overview

In the following, we introduce our heterogeneous temporal GNN-based framework for NIDS, as illustrated in Fig. 2. The framework comprises six key steps: features extraction, relation definition, HTGraph construction, HTGNN model, Adjustable Early Detection Decoder, and heterogeneous knowledge fusion and classification.

First, when presented with a network flow, we record the dynamically updated features as sequences with new packets arriving. These pre-defined features are heterogeneous and have different scales or units. Based on strong relations created for the same type of features (homogeneous features), these homogeneous nodes assemble

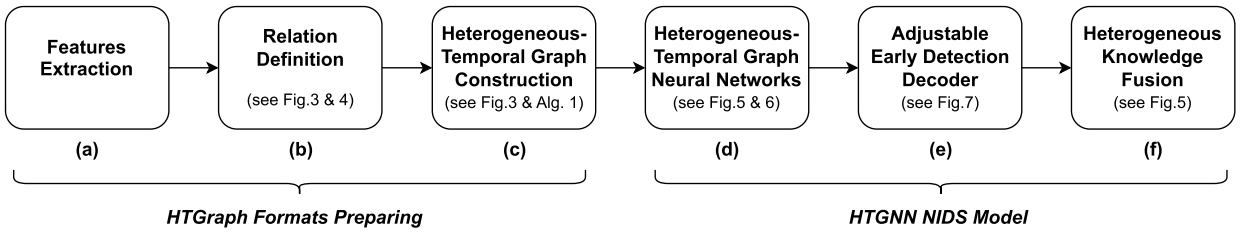

Fig. 2. Our proposed NIDS Framework. It incorporates prior knowledge (features types and correlations) in NIDSs design using HTGraph and HTGNN models.

into a sub-graph. By considering multiple varying features, an HTGraph is generated, which consists of nodes representing heterogeneous features at different times. We also define three types of edges to establish correlation among features: "Heter-Correlation edge", "Homo-Correlation edge" and "Temporal edge".

Subsequently, with the constructed HTGraph, multiple HTGNN layers are designed to generate node (features) embeddings. Corresponding to the three edge types, three types of GNN layers independently aggregate information through each type of edge. During this step, most information is still aggregated among homogeneous neighbors. Finally, we employ the Adjustable Early Detection Decoder to generate embedding for each type of feature. A fusion network is then applied to combine heterogeneous embeddings (learned knowledge) for further prediction.

B. Network Flow HTGraph Construction

Data pre-processing is widely recognized as an essential stage in anomaly detection and is found to predominantly rely on expert domain knowledge for identifying the most relevant parts of network traffic and constructing the initial candidate set of traffic features [6]. Compared with feature vectors, graphs are more flexible, allowing us to create correlations among features and depict features' dynamic updating trends as packets arrive. In this work, we enhance traditional traffic features by representing network flow in the form of an HTGraph. In this section, we introduce the traffic feature set used in this paper and then present the details of constructing an HTGraph with those features.

1) Statistical Features: Feature extraction is a crucial step in all knowledge discovery tasks, including NIDS. From multiple published NIDS datasets, we find that the chosen feature sets differ significantly. Rather than delving into optimizing the feature set to maximize relevance with labels, our focus is on enhancing the effectiveness of a fixed feature set by formatting features in a new structure.

One aspect of improvement involves using a dynamically updated feature sequence instead of a single feature value. We anticipate that the feature sequence can provide more temporal information to better distinguish and classify network traffic as normal or anomalous. Fortunately, extracting the mentioned feature sequence doesn't incur significant computational resources. Because during dataset creation or the practical NIDS detection process, the feature extraction programs, such as NetFlow, Argus, and CICFlowMeter, always update features with new packets arriving. Another aspect is constructing multiple feature sequences into a graph, with details introduced in the following Sec. IV-B2.

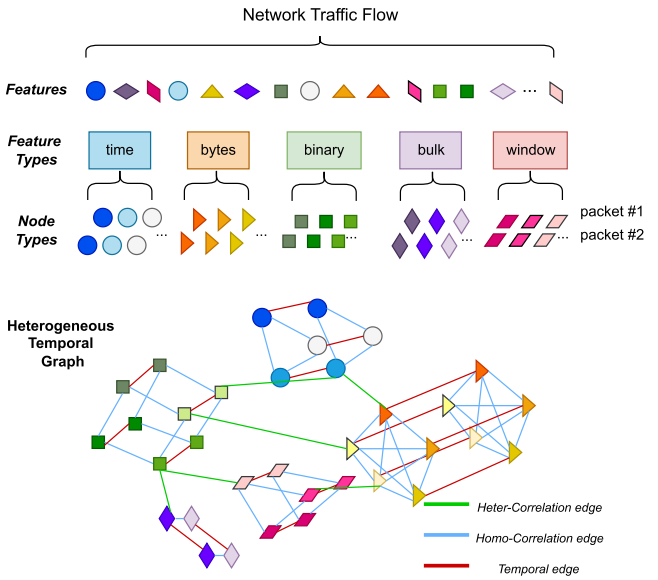

Fig. 3. Heterogeneous temporal graph-based representation of NIDS's feature-based traffic flow. (The feature types used by the CIC-IDS-2017 dataset are used as introducing examples.)

In this work, we utilize the statistical feature set provided in the CIC-IDS-2017 dataset [31], comprising 80 features. These features encompass different attribute types, value scales, and a mix of discrete and continuous variables (heterogeneous). First, because we have already adopted feature sequences instead of a single feature value, the 80 features can be simplified to 28 features by discarding some excess features, such as $tot\_l\_fw\_pkt$, $fw\_pkt\_l\_max$, $fw\_pkt\_l\_min$, $fw\_pkt\_l\_avg$, $fw\_pkt\_l\_std$, which represent total, maximum, minimum, average, and standard deviation size of packets in the forward direction. These are replaced by one feature sequence: "each packet size in the forward direction. Then, based on feature types, we categorize the 28 features into 5 types: "**time**" (3), "**packets**" (8), "**binary**" (9), "**bulk**" (6), and "**window**" (2). The details about statistical features used in this work are shown in Table VIII.

We would like to remark that the chosen feature set is not tied to the method, the features and feature types can all be customized. The utilization of specific features of CIC-IDS-2017 and the following graph construction on those features are just examples to explain the concepts and methods.

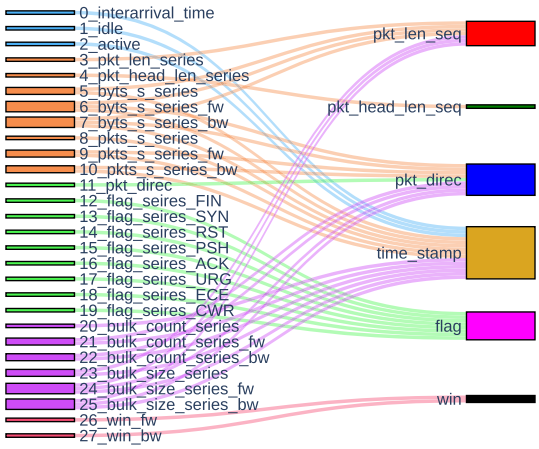

Fig. 4. Heter-correlation edges decided by the usages of basic feature series. (The features used by the CIC-IDS-2017 dataset are used as introducing examples.)

2) HTGraph Construction: As we analyzed before, the challenges posed by heterogeneous and weakly correlated features motivate us to seek a more suitable format for representing network flows. This shift aims to achieve better intrusion detection performance by modeling heterogeneous spatial dependencies among different types of features.

As illustrated in Fig. 3, the extracted feature sequences are transformed into HTGraph structure $HTGraph_{flow}(T)$. Following the definition in Equation (1), the HTGraph consists of four components: V^x, E^{homo}, E^{heter}, E^{temp}.

a) Nodes (V^x): First, we consider the dynamically updated statistical features $(f_1, f_2, \ldots, f_m)_t$ as nodes, and their values are nodes' initial values. Then, based on the feature types, corresponding nodes are divided into x different types. In this work, the number of node types x is 5, which are "**times**", "**packets**", "**binary**", "**bulk**", and "**window**".

b) Homo-correlation edge (E^{homo}): In $HTGraph_{flow}$, the homo-correlation edges are those links between homogeneous nodes (nodes of the same type but representing different features), shown as the blue links in Fig. 3. Although homo-correlation edges share the same color, they actually have different types, which are the same as the connected nodes' type. Indeed homo-correlation edge is represented as $E^{homo} = \{e^x_{(i,j),t} | x \in \{times, packets, binary, bulk, win\}\}$. Because features of the same type have similar value scales or the same unit, they naturally exhibit strong correlations.

c) Heter-correlation edge (E^{heter}): In addition to homogeneous correlation, there are also correlations between heterogeneous features. Heter-correlation edges represent links between heterogeneous nodes (nodes in different types), shown as the green links in Fig. 3. As shown in Fig. 4, we define the heter-correlations based on the shared input components for feature calculation. For calculating the feature series in the left column of Fig. 4, we use 6 basic feature series listed on the right part, which are the packet length series, packet header length series, packet direction series, timestamp series, flag series, and TCP window size series. Heterogeneous feature series sharing the same basic feature series are considered correlated.

d) Temporal edge (E^{temp}): We further take temporal dependency into account. Since we have already extended features to feature sequences, which record the updated features after the packet arrives. Hence, the neighbors in the same features, such as $f^x_{i,t}$ and $f^x_{i,t+1}$, have temporal relation $e^x_{i,(t,t+1)}$, where t is packet arriving sequence number instead of actual time and i is the sequence of features in type x. Therefore, the temporal edge is represented as $E^{temp} = \{e^x_{i,(t,t+1)} | x \in \{times, packets, binary, bulk, win\}\}$. In contrast to the homo-edge and heter-edge, where edge connections show binary direction correlations, temporal edges are unidirectional. Messages can only be shared from the early nodes to late nodes.

Algorithm 1 The Function of Graph Construction

Input: Network traffic $flow = \{pkt_1, pkt_2, \ldots, pkt_T\}$
Output: $HTGraph : (V^x, E^{homo}, E^{heter}, E^{temp})$

function GRAPH CONSTRUCT($flow$)
  for $t = 1\ T$ **do**
    $subflow(t) = \{pkt_1, pkt_2, \ldots, pkt_t\}$
    $(f_1, \ldots, f_m)_t =$ FEATURE EXTRACT($subflow(t)$)
    Split $f_1, \ldots, f_m$ into 5 sets: $\{\cdot\}^{times}, \{\cdot\}^{pkt}, \{\cdot\}^{bin}, \{\cdot\}^{bulk}, \{\cdot\}^{win}$
    for set^x **in** the 5 $sets$ **do**
      for $i = 1\ T\ size(set^x)$ **do**
        $V^x \leftarrow f^x_{i,t} \in set^x$
        $E^{temp} \leftarrow e^x_{i,(t-1,t)}$ to connect $(f^x_{i,t-1}, f^x_{i,t})$
        $E^{homo} \leftarrow e^x_{(i,j),t}$ to connect $(f^x_{i,t}, f^x_{j,t})$
      end for
      for set^y **in** the rest $4sets$ **do**
        for $i = 1\ T\ size(set^x)$, $j = 1\ T\ size(set^y)$ **do**
          if $f^x_{i,t}$, $f^y_{i,t}$ sharing the same basic feature **then**
            $E^{heter} \leftarrow e^{x,y}_{(i,j),t}$ to connect $f^x_{i,t}, f^y_{j,t}$

C. Heterogeneous Temporal Graph Neural Networks

In this section, we formally present our HTGNN designed to address the challenges posed by heterogeneous features and early intrusion detection. HTGNN model comprises two main parts: (1) dynamic features graph embedding; (2) heterogeneous knowledge fusion. Fig. 5 illustrates these two components. The first part extracts node embedding by learning knowledge from the constructed heterogeneous temporal feature graphs. Leveraging the graph structure allows us to model correlations among different features (nodes). In the second part, all node embeddings from each subgraph are aggregated to acquire heterogeneous knowledge for the overall graph classification task, specifically predicting whether the flow is malicious or benign.

1) Dynamic Features Graph Embedding: The key idea of GNN is aggregating information from neighbor nodes through edges. However, as we mentioned in Sec.IV-B2, the nodes' initial values are recorded features, and these features are heterogeneous. Directly aggregating features from different types of nodes poses a challenge for neural networks.

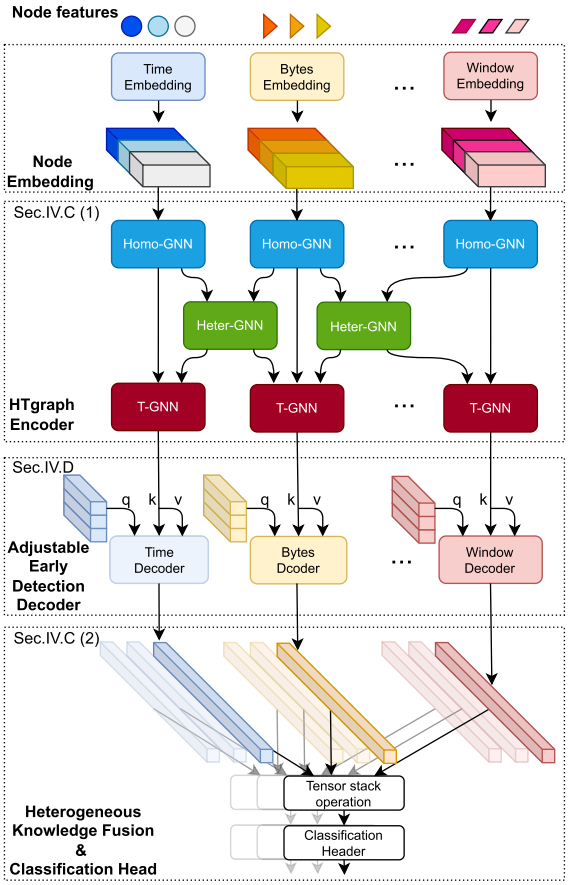

Fig. 5. HTGNN-based NIDS model.

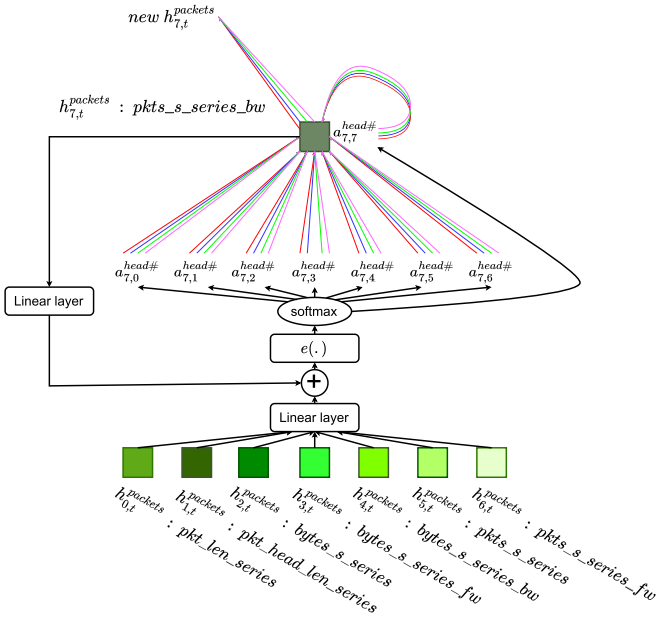

Fig. 6. Multi-head GATv2 in Homo-GNN for "packets" type subgraph, $h^x_{i,t}$ denotes the node embedding feature, which represents the ith statistical feature in x type at time t and $e(.)$ is attention coefficient function. This example shows how node embedding for statistical feature $pkts\_s\_series\_bw$ is updated through multi-head attention. The attentions are learnable weights, which denote the correlations between feature pairs. (The features used by the CIC-IDS-2017 dataset are used as introducing examples.)

To address this issue, we design the dynamic features graph embedding module to encode the heterogeneous features as a fixed-size embedding, which is more suitable for further exchanging among heterogeneous nodes. Therefore, the first stage of homo-encoding only aggregates features through **homo-correlation edges** and **temporal edges** to encode the nodes as same-size vectors. With these vectors, the second stage of heter-encoding incorporates **heter-correlation edges** and aggregates embedding through all three types of edges.

Specifically, we employ independent GATv2 [5] graph neural networks for different types of edges to capture distinct information based on the temporal and semantic dependencies between dynamic features (see Table.III).

$$E^{homo} = \{e^{x}_{(i,j),t}\}$$
$$E^{heter} = \{e^{x_1,x_2}_{(i,j),t}\}$$
$$E^{temp} = \{e^{x}_{i,(t,t+1)}\}$$

a) GATv2: For a given HTGraph's subgraph, defined by a set of edges $\{e\}$ representing a specific knowledge relationship, an independent GATv2 neural network is employed to learn the embedding of feature nodes. For a node in the subgraph, its learned embedding is denoted as $h_{i,t}$. In each GATv2 layer, the embedding $h_{i,t}$ is updated to a new representation $h'_{i,t}$ through adaptive attention aggregation, as described by Equation (4).

$$h'_{i,t} = \mathop{\Big\Vert}_{k=1}^{K} \left(\sum_{j=1}^{n} a^k_{i,j} W^k h_{j,t} \right) \tag{4}$$

where K is a number of heads, j represents the neighbor nodes of node i, t denotes the temporal dimension, $\oplus$ represents concatenation, $W^{x,k}$ is the corresponding learnable weight matrix, $a^{x,k}_j$ are attention coefficients computed using Equation (5).

$$a_{i,j} = \text{softmax}_j\left(e(h_{i,t}, h_{j,t})\right)$$
$$e(h_{i,t}, h_{j,t}) = \mathbf{a}^\top \text{LeakyRelu}(W[h_{i,t} \oplus h_{i,t}]) \tag{5}$$

b) An example of GATv2 for homo-subgraph: For each type of subgraph, which consists of all nodes from the same type, we adopt a multi-head to learn knowledge among homogeneous statistical features to encode corresponding nodes' embedding features. Fig. 6 shows the operation $GATv2_{homo(packets)}$ for the subgraph with "packets" type nodes.

2) Heterogeneous Knowledge Fusion: Following the application of multiple HTGNN layers, the subsequent step is shrinking the sub-graph node embeddings into a sub-graph embedding. One simple alternative is to use the sub-graph pooling layer to generate a comprehensive summary embedding for each distinct feature type. In this work, we employ a Transformer decoder-based module to merge the node embeddings into a summary embedding; more details will be introduced in the following section. This

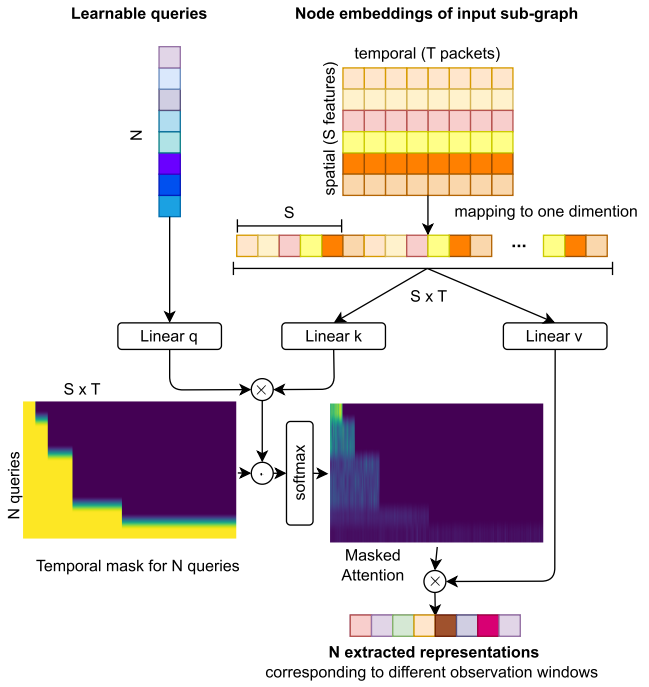

Fig. 7. Adjustable Early Detection Decoder. N independent learnable queries extracted sub-graph representations for intrusion detection. The temporal mask is used for restricting different queries within different observation window sizes.

summary embedding is then directed towards a fusion layer, where it is combined with other types of heterogeneous knowledge to generate the final classification. The process of integrating each distinct feature type into the fusion layer ensures the comprehensive amalgamation of all available knowledge sources, resulting in a more accurate and informed classification outcome.

D. Adjustable Early Detection Decoder

Early intrusion detection poses a challenge as the NIDS receives limited information about rapidly flowing traffic. We define the threshold determining the detection's earliness as the observation window, which dictates the number of packets in the input traffic. With varying observation windows, shifts in the distribution of input traffic data can present a challenge to the generalization of the NIDS model. To adapt the NIDS model's response time after deployment, two key considerations arise (1) the need for an adjustable neural network component to govern the observation window, and (2) ensuring the NIDS model can generalize effectively to varying observation windows.

To fulfill these requirements, we propose an Adjustable Early Detection Decoder designed to extract multiple representations corresponding to different observation windows. As shown in Figure 7, a Transformer Decoder [34] structure is adopted as the required adjustable neural network component. An Adjustable Early Detection Decoder follows each homogeneous sub-graph before the Heterogeneous Knowledge Fusion and classification modules. Within the Adjustable Early Detection Decoder, two inputs are utilized: the learnable queries and the sub-graph node embeddings. Learnable queries are responsible for scoring the importance of different nodes in the input sub-graph relative to the traffic type based on the information in different observation windows. To compute dot-product attention, the node embeddings of the spatial-temporal sub-graph are mapped into one dimension. Subsequently, the dot products of each query vector with all key vectors are calculated. The attention map results from the matrix multiplication between N learnable queries and $S \times T$ keys.

During training, a masking mechanism is employed to ensure that the attention mechanism attends to the nodes within the observation windows. We choose increasing observation windows for different queries so that we can train different queries to extract features for different degrees of early detection. For instance, the first query can only observe the first 8 packets' information, the second from the first 16, and the last one from all. As traffic flows may have a flexible number of packets denoted as T, setting the observation window size uniformly for the i-th query as $\frac{i \times T}{N}, i = 1, 2, \ldots N$ proves challenging due to the unpredictable observation window size for each query. To address this, we choose observation window sizes from powers of 2. In particular, the observation window size for the i th query as

$$ObserWin_i = \begin{cases} 2^j, & \text{if } \dfrac{i \times T}{N} \geq 2^j \text{ and } \dfrac{i \times T}{N} < 2^{j+1} \\ T, & \text{if } i = N. \end{cases} \tag{6}$$

Next, we set the first $S \times ObserWin_i$ elements of the one-dimension mask for the i th query to be 1 and the remaining $S \times (T - ObserWin_i)$ elements to be 0.

Thanks to the Adjustable Early Detection Decoder, the output of the proposed NIDS model for a single traffic flow is extended to N classification results, each corresponding to different observation windows. In the training stage, these multiple results are used to calculate the loss for updating the model at the same time. This implies that the proposed NIDS model is trained to enable different degrees of early detection. After the NIDS model is deployed, users can adjust the degree of early detection by selecting a query without the need to retrain the NIDS model with early detection data.

V. Evaluation and Results

In this section, we present a detailed evaluation of the proposed HTGraph-based traffic flow data format and the corresponding HTGNN-based NIDS model. First, we introduce the experiment setup in Sec. V-A. Then, we compare the proposed method with other existing data formats and the NIDS models in aspects of (1) general detection performance (in terms of accuracy and F1 score et al.) and (2) model transferability against domain change in Sec. V-C. Furthermore, we conduct an ablation study to show the impact of different types of edges in the proposed HTGraph in Sec. V-B. Finally, we evaluate the early detection performance to show the proposed HTGNN-based NIDS model is generalized to varying intrusion detection response times in Sec. V-E.

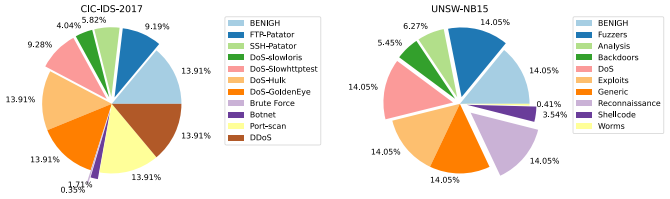

Fig. 8. The ratio of each class in the resampled CIC-IDS-2017 and UNSW-NB15 datasets. For each class, we choose up to 6000 samples, which have more packets in flow.

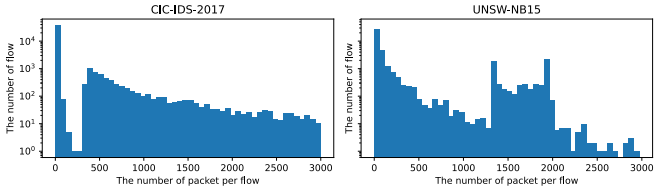

Fig. 9. The distribution of the packet number per flow for the CIC-IDS-2017 and UNSW-NB15 datasets.

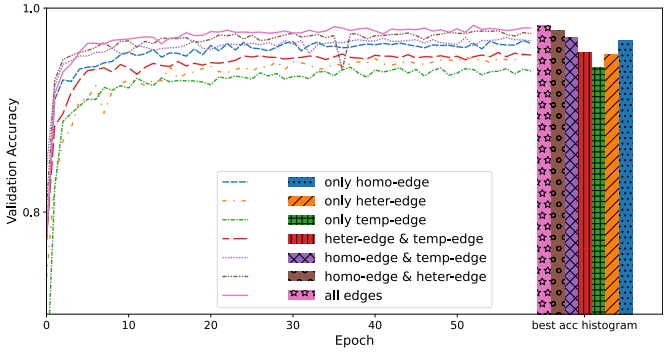

Fig. 10. Validation accuracy with various edges combinations on CIC-IDS-2017.

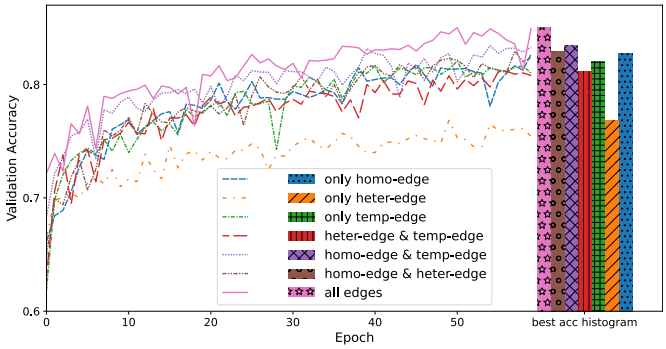

Fig. 11. Validation accuracy with various edges combinations on UNSW-NB15.

A. Experiment Setup

Experimental Environment. To train models, we use a high-performance computer running Ubuntu 18.04 on 3.30GH Intel(R) Core(TM) i9-9820X CPU with 128 GB main memory equipped with two NVIDIA GeForce RTX 4070 Ti GPUs. All models are built with Pytorch 2.0.1 and PyG [11] 2.3.0, then are trained on two GPUs by using the DistributedDataParallel module.

CIC-IDS-2017 dataset is made up of benign flows and 14 classes of malicious flows. For our paper, we used the improved version of CIC-IDS-2017 as described in [9]. Due to the limited number of samples available, we excluded four malicious classes from our analysis: Heartbleed (11 samples), Web Attack-XSS (27 samples), Web Attack-SQL Injection (12 samples), and Infiltration (32 samples). Therefore, in our experiment, the training and testing data comprised a total of 11 classes, 1 for benign flows and 10 for malicious flows.

UNSW-NB15 dataset [26] consists of benign flows and 9 classes of malicious flows, which include Fuzzers, Analysis, Backdoors, DoS, Exploits, Generic, Reconnaissance, Shellcode, and Worms. The hybrid of real modern normal activities and synthetic contemporary attack behaviors are generated by the IXIA PerfectStorm tool.

Evaluation Metrics The intrusion detection task in this work is an imbalance multi-class classification task. Therefore, we evaluate the overall intrusion detection performance in terms of weighted Accuracy (ACC), weighted Detection Rate (DR), weighted False alarm rate (FAR), and weighted F1 score (F1). The "weighted" refers to calculating the mean of the metrics for each class, with consideration given to each class's proportion of a class's support to the total support across all classes.

In this work, we use and re-process the raw Pcap data from both CIC-IDS-2017 and UNSW-NB15 datasets, for three main purposes: (1) constructing the proposed graph data format to represent network traffic flow; (2) comparing it with other existing formats, such as tabular data, and feature series; (3) studying the early detection problem. For the graph and feature series formats, we extract the dynamic feature series defined in Table VIII. As for the tabular format, we adopt the feature set defined in the CIC-IDS-2017 dataset, which is also the comprehensive version of the feature set in Table VIII. The classes' sampling proportions on two datasets are reported by Fig. 8. The distributions of the packet number per flow for two datasets are reported by Fig. 9.

B. Impact of Different Edges

To analyze the effectiveness of our heterogeneous temporal graph format design, we conduct an ablation study comparing variant combinations of different types of edge connections. Since changes in edge connections do not affect the HTGNN structure but influence the message passing through graph edges, we evaluated the impact of different edges by training the same HTGNN structure from scratch with different edge connections.

Fig. 10 and Fig. 11 present validation accuracy curves during training on CIC-IDS-2017 and UNSW-NB15, respectively. Clearly, applying all types of edge has the best performance across different datasets. When considering only using one type of edge, the homo-edge has more impact on model training than the heter-edge and temp-edge on both datasets. Moreover, using only the homo-edge connection can achieve even better performance than the combined heter-edge and temp-edge. In addition, for all edge combinations, if there is a homo-edge, they all have better performance than the combinations without a homo-edge.

TABLE IV
COMPARISON RESULT WITH EXISTING TABULAR AND FEATURE SERIES FORMATS-BASED METHODS

Formats	Models	CIC-IDS-17 ACC	CIC-IDS-17 DR	CIC-IDS-17 FAR	CIC-IDS-17 F1	UNSW-NB15 ACC	UNSW-NB15 DR	UNSW-NB15 FAR	UNSW-NB15 F1
Tabular	LuNet	0.9192	0.9243	0.0302	0.92	0.7925	0.8003	0.0104	0.7972
	FT-Transformer	**0.9913**	**0.9914**	**0.0041**	**0.9914**	**0.8613**	**0.8655**	**0.0072**	**0.8627**
Fseries	Lucid	0.9789	0.9792	0.0041	0.9788	0.8041	0.8041	0.0098	0.8022
	Informer	0.9834	0.9835	0.0048	0.9834	0.8213	0.8217	0.0094	0.8188
Graph	HTGNN+1 query (our)	**0.9827**	**0.9829**	**0.0043**	**0.9828**	**0.8532**	**0.8555**	**0.0075**	**0.8513**
	HTGNN+8 queries (our)	**0.9934**	**0.9935**	**0.0025**	**0.9934**	**0.8612**	**0.8632**	**0.0070**	**0.8612**

The performance difference on the two datasets is notable: the heter-edge-only case outperforms the temp-edge-only case on CIC-IDS-2017 but performs worse than the temp-edge-only case on UNSW-NB15. A similar situation also happens with the two combinations of homo-edge plus heter-edge and homo-edge plus temp-edge. Therefore, the results show the heter-edge has a larger impact on CIC-IDS-2017, while the temp-edge has a larger impact on UNSW-NB15. However, the homo-edge consistently has the most significant impact. The lesser impact of heter-edge and temp-edge is attributed to other components, such as AE Decoder and HK Fusion modules, to some degree play the same role as fusing the heterogeneous and temporal features.

C. Detection Performance

In this section, we compare the detection performance among three different traffic representation formats, which are tabular, feature series, and the proposed graph. For both tabular and feature series formats, we compare with one specific NIDS model and one state-of-the-art DL model corresponding to the format.

Table IV illustrates the general intrusion detection performance comparisons on datasets CIC-IDS-2017 and UNSW-NB15. Overall, almost all formats and models achieved nearly perfect performance (ACC) on CIC-IDS-2017 but exhibited relatively poorer performance (around 0.8 ACC) on UNSW-NB15.

On the CIC-IDS-2017 dataset, the proposed HTGNN model achieved accuracies of 0.9827 and 0.9934 (the highest) when utilizing 1 query and 8 queries in the AE Decoder, respectively. The two Transformer-based methods, FT-transformer (for tabular) and Informer (for feature series), both slightly outperform the 1 query case HTGNN but lagged behind its performance with 8 queries. Among other NIDS models, the Lucid model, based on Convolutional Neural Networks (CNN) for feature series, and the LuNet model, incorporating both CNN and LSTM for tabular format, were also considered. On the UNSW-NB15 dataset, the proposed HTGNN model achieved accuracies of 0.8532 and 0.8612 with 1 query and 8 queries in the AE Decoder, respectively. The FT-transformer model achieves 0.8613 accuracy and 0.8627 F1 score (the highest) slightly outperforming both the 1 query and 8 queries HTGNN.

From Table IV, three main observations emerge: (1) the proposed HTGNN model, utilizing the HTGraph format, achieves comparable general classification performance with state-of-the-art DL models for both tabular and feature series formats; (2) employing multiple queries in the AE Decoder enhances general performance beyond the 1 query baseline; (3) self-attention-based methods (FT-transformer, Informer, and our HTGNN using graph attention and Transformer Decoder) outperform CNN or LSTM-based methods in general performance.

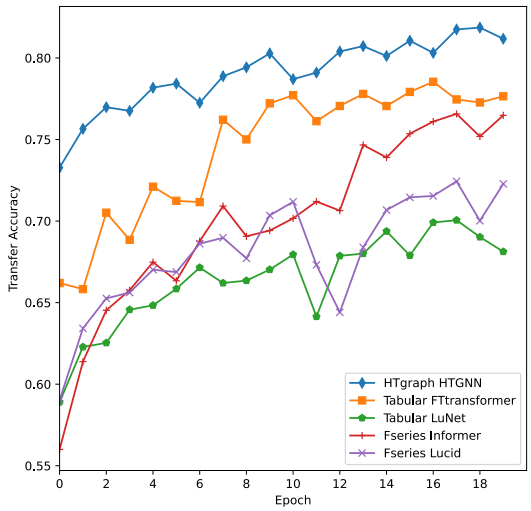

Fig. 12. The validation accuracy curves when re-training the last layer of CIC-IDS-17 models on UNSW-NB15 for 20 epochs.

D. Domain Adaptation Performance

To assess the transferability of the trained models, we conducted a domain adaptation task, specifically evaluating the ability to transfer models trained solely on CIC-IDS-2017 to UNSW-NB15. Since Table IV indicates that the CIC-IDS-2017 dataset is comparatively easier than UNSW-NB15, we focused on the domain adaptation task from CIC-IDS-2017 to UNSW-NB15 (from an easier to a more challenging dataset) to showcase model transferability. We did not explore the reverse case. Throughout the transfer learning process, we maintained frozen parameters for all models, except for their last dense layers.

Fig. 12 shows validation accuracy curves during the retraining of the output layers for all models listed in Table IV on the UNSW-NB15 dataset for 20 epochs. The combination of HTGraph format and HTGNN model demonstrates quicker adaptation to the new dataset compared to the other data formats and models. In Fig. 12, the HTGraph+HTGNN curve exhibits a significantly superior starting point, with around 0.73 accuracy for the first epoch, in contrast to

TABLE V

DOMAIN ADAPTATION PERFORMANCE COMPARISON. ALL MODELS HAVE BEEN TRAINED ON THE CIC-IDS-17. THEN WE RE-TRAIN THE LAST LAYER ON THE UNSW-NB15 AND FREEZE THE REST LAYERS. THE ARROWS SHOW THE DEGRADED PERFORMANCE COMPARING WITH THE TABLE IV

Formats	Models	ACC	DR	FAR	F1
Tabular	LuNet	0.7113 (8.12% ↓)	0.7005 (9.98% ↓)	0.0111 (0.07% ↑)	0.6991 (9.81% ↓)
	FT-Transformer	0.8255 (3.58% ↓)	0.8281 (3.74% ↓)	0.0078 (0.06% ↑)	0.8225 (4.02% ↓)
Fseries	Lucid	0.7263 (7.78% ↓)	0.7358 (6.83% ↓)	0.0156 (0.58% ↑)	0.7228 (7.94% ↓)
	Informer	0.7692 (5.21% ↓)	0.7668 (5.49% ↓)	0.0107 (0.13% ↑)	0.7657 (5.31% ↓)
Graph	HTGNN with 1 query (our)	**0.8427 (1.05% ↓)**	**0.8431 (1.24% ↓)**	**0.0077 (0.02% ↑)**	**0.8423 (0.9% ↓)**
	HTGNN with 8 queries (our)	**0.8601 (1.23% ↓)**	**0.8622 (1.03% ↓)**	**0.0074 (0.04% ↑)**	**0.8601 (1.23% ↓)**

TABLE VI

SYMMETRICAL PAIRS EXPERIMENTS TRAIN AND TEST NIDS MODELS WITH THE SAME OBSERVATION WINDOW SIZE TO SHOW THE MODEL CAPABILITY OF EARLY DETECTION

		$ObserWins$ (CIC-IDS-17)				$ObserWins$ (UNSW-NB15)			
Formats	**Models**	**16**	**64**	**256**	**1024**	**16**	**64**	**256**	**1024**
Tabular	LuNet	0.9050	0.9093	0.9148	0.9192	0.6529	0.7009	0.7237	0.7925
	FT-Transformer	0.9882	0.9881	0.9903	0.9913	0.7705	0.8238	0.8556	0.8613
Fseries	Lucid	0.9647	0.9663	0.9775	0.9789	0.7033	0.7226	0.7485	0.8041
	Informer	0.9776	0.9790	0.9806	0.9834	0.7438	0.7887	0.8064	0.8213
Graph	HTGNN with 1 query (our)	0.9726	0.9745	0.9782	0.9827	0.7627	0.7899	0.8235	0.8532
	HTGNN with 8 queries (our)	0.9920	0.9917	0.9930	0.9934	0.8272	0.8396	0.8507	0.8612

TABLE VII

ASYMMETRICAL PAIRS EXPERIMENTS TRAIN AND TEST NIDS MODELS WITH DIFFERENT OBSERVATION WINDOW SIZES TO SHOW THE MODEL GENERALIZATION TO EARLY DETECTION

		$ObserWins$ (CIC-IDS-17)				$ObserWins$ (UNSW-NB15)			
Formats	**Models**	**16**	**64**	**256**	**1024**	**16**	**64**	**256**	**1024**
Tabular	LuNet	0.5564	0.5579	0.5653	0.9192	0.5496	0.5836	0.6033	0.7925
	FT-Transformer	0.8976	0.9131	0.9469	0.9913	0.6775	0.6844	0.7181	0.8613
Fseries	Lucid	0.7497	0.8051	0.8091	0.9789	0.4951	0.5241	0.5303	0.8041
	Informer	0.9341	0.9657	0.9722	0.9834	0.6508	0.7296	0.7477	0.8213
Graph	HTGNN with 1 query (our)	0.8871	0.9041	0.9273	0.9827	0.7072	0.7647	0.7897	0.8532
	HTGNN with 8 queries (our)	0.9812	0.9929	0.9938	0.9934	0.7941	0.8473	0.8482	0.8612

the Tabular+FT-Transformer, which starts around 0.66. The HTGraph+HTGNN consistently outperforms other methods throughout the retraining process.

We show the details of the final transfer learning performances and the relevant performance decline, compared with direct training on UNSW-NB15 in Table V. Besides the proposed methods, we observed the other self-attention-based methods also exhibit good transferability. We attribute the robust transferability of self-attention-based models to the mechanism's focus on modeling correlations among various input items, such as the features in the tabular format. This emphasizes the importance of learning relationships among input items rather than absolute value scales. Moreover, the prior knowledge-guided HTGraph format and the associated HTGNN model help streamline the learning scope of these relations, contributing to enhanced transferability.

E. Early Detection Evaluation

In the early detection evaluation experiments, we assess two key aspects: the model's capability for early detection and its generalization to early detection scenarios. To evaluate the model's early detection capability, we employ symmetrical pairs of dataset versions with different observation window sizes (i.e., 16, 64, 256) for training and testing NIDS models. To assess the model's generalization to early detection, we utilize asymmetrical pairs of dataset versions.

In the initial evaluation, we explore the NIDS models' early detection capability by reducing both training and testing observation window sizes. In this symmetrical case, less temporal information can be observed by the NIDS models which cause moderate performance degradation on both CIC-IDS-2017 and UNSW-NB15 datasets across all formats and models in Table VI. Notably, there is only around a 1% accuracy degradation on CIC-IDS-2017 when the observation window sizes are reduced from 1024 to 16.

Combining the findings on edge effectiveness presented in Fig.10, we assume the CIC-IDS-2017 should have some significant patterns that are independent of temporal information. Hence, the temp-edge has the least impact in Fig. 10, and the reduction in observation window sizes causes only a slight degradation, as indicated in Table VI. Moreover, the decreasing trends in UNSW-NB15 are more pronounced than those in CIC-IDS-2017. This disparity is also evident in Fig. 11, where the temp-edge has a more significant impact on the UNSW-NB15 dataset. The proposed HTGNN method, especially when using 8 queries in the AE Decoder, can maintain its performance on both datasets, and the 1 query case is also comparable to the FT-Transformer. Therefore,

TABLE VIII
STASTICAL FEATURES

Feature Type	Feature name	Description
Time	flow_interarrival_time	packet inter-arrival time
	idle	time duration of a flow was idle before becoming active
	active	time duration of a flow was active before becoming idle
Packets	pkt_len_series	packet length of a flow
	pkt_head_len_series	packet's header length of a flow
	byts_s_series	packet byte rate (bytes of per second) transferred in both directions
	byts_s_series_fw	packet byte rate (bytes of per second) transferred per second in the forward direction
	byts_s_series_bw	packet byte rate (bytes of per second) transferred per second in the backward direction
	pkts_s_series	packet rate (in packets per second) transferred in both directions
	pkts_s_series_fw	packet rate (in packets per second) transferred in the forward direction
	pkts_s_series_bw	packet rate (in packets per second) transferred in the backward direction
Binary	pkt_direc	packet transferred direction
	flag_seires_FIN	arriving packet is FIN or not
	flag_seires_SYN	arriving packet is SYN or not
	flag_seires_RST	arriving packet is RST or not
	flag_seires_PSH	arriving packet is PSH or not
	flag_seires_ACK	arriving packet is ACK or not
	flag_seires_URG	arriving packet is URG or not
	flag_seires_ECE	arriving packet is ECE or not
	flag_seires_CWR	arriving packet is CWR or not
Bulk	bulk_rate_series	bulk rate (in bulks per second) in both directions
	bulk_rate_series_fw	bulk rate (in bulks per second) in the forward direction
	bulk_rate_series_bw	bulk rate (in bulks per second) in the backward direction
	bulk_size_series	bulk rate (in bytes) in both directions
	bulk_size_series_fw	bulk rate (in bytes) in the forward direction
	bulk_size_series_bw	bulk rate (in bytes) in the backward direction
Window	win_fw	number of bytes sent in the initial window in the forward direction
	win_bw	number of bytes sent in the initial window in the backward direction

we believe the proposed HTGNN exhibits good early detection capability, and the AE Decoder can further enhance this capability.

In the asymmetrical case, we assess the models' generalization to early versions of input data. All models are trained with a 1024 observation window size on both CIC-IDS-2017 and UNSW-NB15, and their baseline performance is presented in Table IV. As the observation window sizes of training and testing data differ (asymmetrical), the trends of decreasing detection accuracy results across all formats and models become more pronounced than in the symmetrical case. The results demonstrate that even though training and testing data have different observation window sizes, the proposed HTGNN methods exhibit greater generalization to distribution shifts caused by the decreasing observation window size compared to other methods.

F. Discussion on Computational Complexity

Compared with the Tabular and Feature Series formats, the proposed HTGraph format obviously will introduce additional computation for building the graph structure. In our experiments, our graph has $N \times T$ nodes, where $N = 28$ is the number of features and T is the number of packets, the number of temporal edges is $28 \times (T-1)$, the number of home-edges is $(3^2 + 8^2 + 9^2 + 6^2 + 2^2) \times T$, the number of heter-edges is $(4 \times 3 + 4 \times 1 \times 4 + 3 \times 5 \times 6) \times T$ (, based on Fig. 4). Therefore, in the case of early detection where T is relevant small, the graph will not increase much computational complexity. In addition, the home-edges and heter-edges in each timestamp can also be pre-calculated.

VI. CONCLUSION

In this paper, we present a novel approach to incorporating prior knowledge into the design of early and transferable NIDS. Our approach involves leveraging a heterogeneous temporal graph format that encodes prior knowledge about NIDS feature types and their interdependence. Additionally, we propose a corresponding graph neural network model, called HTGNN, capable of classifying network flows in the HTGraph format. An Adjustable Early Detection Decoder is introduced to further enhance generalization to early detection. Our experiments on the CIC-IDS-2017 and UNSW-NB15 datasets demonstrate that our proposed method achieves comparable performance (98.27% and 85.32%) with state-of-the-art deep learning methods in terms of accuracy and F1 score et al. At the same time, our proposed method demonstrates enhanced transferability. When adapting the model trained on CIC-IDS-2017 data to the UNSW-NB15 dataset—accomplished by retraining only the last layer for 20 epochs—there's only a 1.05% drop. This improvement is observed in domain adaptation tasks and a better ability to generalize to changes in input distribution caused by early detection. Notably, the dynamic packet number of traffic does not significantly impact performance. In future research, we aim to collaborate the proposed traffic flow-level (features) graph with the existing network topology graph-based NIDSs. In the future collaborating graph NIDS, the proposed K-GetNID is suitable for distributed deployment in different nodes in the whole network topology (based on its transferability) and can efficiently extract traffic embeddings (based on its early detection ability).

CIC-IDS-2017		UNSW-NB15	
Class	not look number	Class	not look number
Benign	4845.93	Benign	1647.18
FTP-Patator	27.94	Fuzzers	39.58
SSH-Patator	54.97	Analysis	15.09
DoS slowloris	16.66	Backdoors	23.9
DoS Slowhttptest	14.57	DoS	66.84
DoS Hulk	17.98	Exploits	194.12
DoS GoldenEye	14.94	Generic	67.98
Brute Force	151.9	Reconnaissance	11.47
Bot	13.43	Shellcode	8.62
PortScan	13.18	Worms	85.12
DDoS	16.17		

VII. Appendix

NIDS	Network Intrusion Detection System.
APT	Advanced and Persistent Threat.
ML	Machine Learning.
DL	Deep Learning.
HTGraph	Heterogeneous Temporal Graph.
HTGNN	Heterogeneous Temporal Graph Neural Network.
MLP	Multi-Layer Perceptron.
LSTM	Long Short-Term Memory.
CNN	Convolutional Neural Networks.
TIG	Traffic Interaction Graph.
DLGNN	Dynamic Line Graph Neural Network.
ACC	Accuracy.
DR	Detection Rate.
FAR	False alarm rate.

References

[1] G. Apruzzese et al., "The role of machine learning in cybersecurity," *Digital Threats, Res. Pract.*, vol. 4, no. 1, pp. 1–38, Mar. 2023.

[2] V. Borisov, T. Leemann, K. Seßler, J. Haug, M. Pawelczyk, and G. Kasneci, "Deep neural networks and tabular data: A survey," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 35, no. 6, pp. 7499–7519, Jun. 2024.

[3] G. Bovenzi, G. Aceto, D. Ciuonzo, A. Montieri, V. Persico, and A. Pescapé, "Network anomaly detection methods in IoT environments via deep learning: A fair comparison of performance and robustness," *Comput. Secur.*, vol. 128, May 2023, Art. no. 103167.

[4] G. Bovenzi, G. Aceto, D. Ciuonzo, V. Persico, and A. Pescapé, "A hierarchical hybrid intrusion detection approach in IoT scenarios," in *Proc. IEEE Global Commun. Conf. (GLOBECOM)*, Taipei, Taiwan, May 2020, pp. 1–7.

[5] S. Brody, U. Alon, and E. Yahav, "How attentive are graph attention networks?" 2021, *arXiv:2105.14491*.

[6] J. J. Davis and A. J. Clark, "Data preprocessing for anomaly based network intrusion detection: A review," *Comput. Secur.*, vol. 30, nos. 6–7, pp. 353–375, 2011.

[7] R. Doriguzzi-Corin, S. Millar, S. Scott-Hayward, J. Martínez-del-Rincón, and D. Siracusa, "LUCID: A practical, lightweight deep learning solution for DDoS attack detection," *IEEE Trans. Netw. Service Manag.*, vol. 17, no. 2, pp. 876–889, Jun. 2020.

[8] G. Duan, H. Lv, H. Wang, and G. Feng, "Application of a dynamic line graph neural network for intrusion detection with semisupervised learning," *IEEE Trans. Inf. Forensics Security*, vol. 18, pp. 699–714, 2023.

[9] G. Engelen, V. Rimmer, and W. Joosen, "Troubleshooting an intrusion detection dataset: The CICIDS2017 case study," in *Proc. IEEE Secur. Privacy Workshops (SPW)*, May 2021, pp. 7–12.

[10] Y. Fan et al., "Heterogeneous temporal graph transformer: An intelligent system for evolving Android malware detection," in *Proc. 27th ACM SIGKDD Conf. Knowl. Discovery Data Mining*, Aug. 2021, pp. 2831–2839.

[11] M. Fey and J. E. Lenssen, "Fast graph representation learning with PyTorch geometric," in *Proc. ICLR Workshop Represent. Learn. Graphs Manifolds*, 2019, pp. 1–9.

[12] Y. Gorishniy, I. Rubachev, V. Khrulkov, and A. Babenko, "Revisiting deep learning models for tabular data," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 34, 2021, pp. 18932–18943.

[13] I. Guarino, G. Bovenzi, D. Di Monda, G. Aceto, D. Ciuonzo, and A. Pescapé, "On the use of machine learning approaches for the early classification in network intrusion detection," in *Proc. IEEE Int. Symp. Meas. Netw. (MN)*, Jul. 2022, pp. 1–6.

[14] X. Han, T. Pasquier, A. Bates, J. Mickens, and M. Seltzer, "UNICORN: Runtime provenance-based detector for advanced persistent threats," 2020, *arXiv:2001.01525*.

[15] H. He, X. Sun, H. He, G. Zhao, L. He, and J. Ren, "A novel multimodal-sequential approach based on multi-view features for network intrusion detection," *IEEE Access*, vol. 7, pp. 183207–183221, 2019.

[16] X. Hu, W. Gao, G. Cheng, R. Li, Y. Zhou, and H. Wu, "Toward early and accurate network intrusion detection using graph embedding," *IEEE Trans. Inf. Forensics Security*, vol. 18, pp. 5817–5831, 2023.

[17] M. Jin et al., "A survey on graph neural networks for time series: Forecasting, classification, imputation, and anomaly detection," 2023, *arXiv:2307.03759*.

[18] J. Khoury, D. Klisura, H. Zanddizari, G. D. L. T. Parra, P. Najafirad, and E. Bou-Harb, "Jbeil: Temporal graph-based inductive learning to infer lateral movement in evolving enterprise networks," in *Proc. IEEE Symp. Secur. Privacy (SP)*, Oct. 2023, p. 9.

[19] T. Liu, Z. Qian, J. Berrevoets, and M. van der Schaar, "Goggle: Generative modelling for tabular data by learning relational structure," in *Proc. Int. Conf. Learn. Represent.*, 2023, pp. 1–12.

[20] W. W. Lo, S. Layeghy, M. Sarhan, M. Gallagher, and M. Portmann, "E-GraphSAGE: A graph neural network based intrusion detection system for IoT," in *Proc. IEEE/IFIP Netw. Operations Manag. Symp.*, Budapest, Hungary, Apr. 2022, pp. 1–9.

[21] M. López-Vizcaíno, F. J. Novoa, D. Fernández, V. Carneiro, and F. Cacheda, "Early intrusion detection for OS scan attacks," in *Proc. IEEE 18th Int. Symp. Netw. Comput. Appl. (NCA)*, Sep. 2019, pp. 1–5.

[22] W. Luo et al., "Dynamic heterogeneous graph neural network for real-time event prediction," in *Proc. 26th ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining*, Aug. 2020, pp. 3213–3223.

[23] P. Manso, J. Moura, and C. Serr ao, "SDN-based intrusion detection system for early detection and mitigation of DDoS attacks," *Information*, vol. 10, no. 3, p. 106, Mar. 2019.

[24] S. M. Milajerdi, R. Gjomemo, B. Eshete, R. Sekar, and V. Venkatakrishnan, "HOLMES: Real-time APT detection through correlation of suspicious information flows," in *Proc. IEEE Symp. Security Privacy (SP)*, May 2019, pp. 1137–1152.

[25] Y. Mirsky, T. Doitshman, Y. Elovici, and A. Shabtai, "Kitsune: An ensemble of autoencoders for online network intrusion detection," in *Proc. Netw. Distrib. Syst. Secur. Symp.*, 2018, pp. 1–15.

[26] N. Moustafa and J. Slay, "The evaluation of network anomaly detection systems: Statistical analysis of the UNSW-NB15 data set and the comparison with the KDD99 data set," *Inf. Secur. J., Global Perspective*, vol. 25, nos. 1–3, pp. 18–31, Apr. 2016.

[27] E. Grumbling and A. Johnson, *Implications of Artificial Intelligence for Cybersecurity: Proceedings of a Workshop*. Washington, DC, USA: National Academies Press, 2020.

[28] R. Paudel and H. H. Huang, "Pikachu: Temporal walk based dynamic graph embedding for network anomaly detection," in *Proc. NOMS IEEE/IFIP Netw. Oper. Manag. Symp.*, Apr. 2022, pp. 1–7.

[29] M. Pivarníková, P. Sokol, and T. Bajtoš, "Early-stage detection of cyber attacks," *Information*, vol. 11, no. 12, p. 560, Nov. 2020.

[30] E. Quiring et al., "DoS and don'ts of machine learning in computer security," in *Proc. 31st USENIX Secur. Symp.*, Boston, MA, USA, 2022, pp. 3971–3988.

[31] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, "Toward generating a new intrusion detection dataset and intrusion traffic characterization," in *Proc. ICISSp*, vol. 1, 2018, pp. 108–116.

[32] M. Shen, J. Zhang, L. Zhu, K. Xu, and X. Du, "Accurate decentralized application identification via encrypted traffic analysis using graph neural networks," *IEEE Trans. Inf. Forensics Security*, vol. 16, pp. 2367–2380, 2021.

[33] R. Sommer and V. Paxson, "Outside the closed world: On using machine learning for network intrusion detection," in *Proc. IEEE Symp. Security Privacy*, May 2010, pp. 305–316.

[34] A. Vaswani et al., "Attention is all you need," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 30, 2017, pp. 1–11.

[35] M. Wang, N. Yang, D. H. Gunasinghe, and N. Weng, "On the robustness of ML-based network intrusion detection systems: An adversarial and distribution shift perspective," *Computers*, vol. 12, no. 10, p. 209, Oct. 2023.
[36] M. Wang, N. Yang, and N. Weng, "Exploring the impact of early detection on DL-based NIDSs models," in *Proc. IEEE 14th Annu. Ubiquitous Comput., Electron. Mobile Commun. Conf. (UEMCON)*, Oct. 2023, pp. 0684–0691.
[37] M. Wang, N. Yang, and N. Weng, "Securing a smart home with a transformer-based IoT intrusion detection system," *Electronics*, vol. 12, no. 9, p. 2100, May 2023.
[38] P. Wu and H. Guo, "LuNet: A deep neural network for network intrusion detection," in *Proc. IEEE Symp. Ser. Comput. Intell. (SSCI)*, Jul. 2019, pp. 617–624.
[39] C. Xu, J. Shen, and X. Du, "A method of few-shot network intrusion detection based on meta-learning framework," *IEEE Trans. Inf. Forensics Security*, vol. 15, pp. 3540–3552, 2020.
[40] R. Xu, G. Wu, W. Wang, X. Gao, A. He, and Z. Zhang, "Applying self-supervised learning to network intrusion detection for network flows with graph neural network," *Comput. Netw.*, vol. 248, Jun. 2024, Art. no. 110495.
[41] Y. Yao, L. Su, Z. Lu, and B. Liu, "STDeepGraph: Spatial–temporal deep learning on communication graphs for long-term network attack detection," in *Proc. 18th IEEE Int. Conf. Trust, Secur. Privacy Comput. Commun./13th IEEE Int. Conf. Big Data Sci. Eng. (TrustCom/BigDataSE)*, Aug. 2019, pp. 120–127.
[42] C. Zhang, D. Song, C. Huang, A. Swami, and N. V. Chawla, "Heterogeneous graph neural network," in *Proc. 25th ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining*, 2019, pp. 793–803.
[43] Y. Zhang, X. Chen, L. Jin, X. Wang, and D. Guo, "Network intrusion detection: Based on deep hierarchical network and original flow data," *IEEE Access*, vol. 7, pp. 37004–37016, 2019.
[44] H. Zhou et al., "Informer: Beyond efficient transformer for long sequence time-series forecasting," in *Proc. AAAI Conf. Artif. Intell.*, vol. 35, 2021, pp. 11106–11115.
[45] F. Zola, L. Segurola-Gil, J. L. Bruse, M. Galar, and R. Orduna-Urrutia, "Network traffic analysis through node behaviour classification: A graph-based approach with temporal dissection and data-level preprocessing," *Comput. Secur.*, vol. 115, Apr. 2022, Art. no. 102632.

Minxiao Wang received the B.S. degree in electronic and communication engineering and the M.S. degree in electronic and information engineering from the Civil Aviation University of China in 2014 and 2018, respectively. He is currently pursuing the Ph.D. degree with the School of Electrical, Computer, and Biomedical Engineering, Southern Illinois University Carbondale, Carbondale, IL, USA. His research interests include developing machine learning and deep learning models for diverse applications related action recognition, object detection, and network and intrusion detection.

Ning Yang (Member, IEEE) received the M.S. degree in computer engineering from the University of Massachusetts at Amherst, USA, in 2006, and the Ph.D. degree in computer engineering from Southern Illinois University Carbondale, USA, in 2020. She is currently an Assistant Professor with the School of Computing, Information Technology Program, Southern Illinois University Carbondale. Her research interests include network security, the Internet of Things, and machine learning.

Ning Weng (Senior Member, IEEE) received the B.S. degree in electrical engineering and in electrical and computer engineering from Huazhong University of Science and Technology, China, in 1996, the M.S. degree in electrical and computer engineering from the University of Central Florida, Orlando, in 2000, and the Ph.D. degree in electrical and computer engineering from the University of Massachusetts, Amherst, in 2005. He is currently a Full Professor with the School of Electrical, Computer, and Biomedical Engineering, Southern Illinois University Carbondale. He is engaged in research and teaching in the areas of computer networks and security. His research interests include scalable system design for deep packet inspection and many-field packet classification, quality of information modeling, and deep leaning models for network intrusion detection. He has been active as a Program Committee Member of several professional conferences, including IEEE INFOCOM and ACM SAC. At Southern Illinois University, he has received three times for Outstanding Teacher Award of the Department of Electrical and Computer Engineering.